Promises in JS

Lec-14

Made with GAMMA

Asynchronous Promises

- A Promise represents an **future result of an asynchronous operation**.
- It acts as a placeholder for a value not that is not available yet.
- **A Promise does not make code asynchronous**, It represents the **result of an asynchronous operation.**
- When an operation is finished, **Promise is either fulfilled with a value or rejected with an error**.
- Every promise is always in one of the three states:
    - **Pending**: The Operation has started but not finished.
    - **Fulfilled**: The Operation completed succesfully.
    - **Rejected**: The Opearation failed.



- **A promise is settled when it is fulfilled or rejected**.

Creating a Promise

- We can **create** promise using **promise constructor**
- **Syntax:**

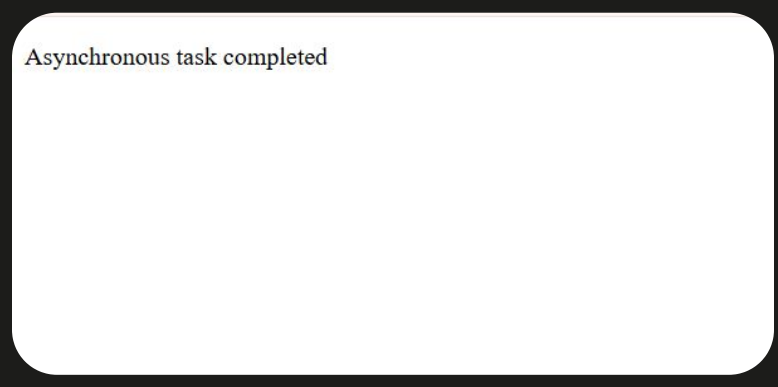
```
const promise = new Promise(function(resolve, reject))
```

- The Constructor **receives a callback function** with two parameters.
- **resolve**: Fulfills the promise with a value.
- **reject**: Rejects the promise with an error.

> • **Note: Call either resolve or reject, a promise can be settled only once**.

- **Eg to create a promise:**

```
const newPromise = new Promise (function(resolve, reject){
  setTimeout(() => {
      document.body.innerHTML = 'Async task completed'
   }, 2000);
})
```



Made with GAMMA

Asynchronous Promises

Using a Promise

.then()

- If the Promise is fulfilled, we can handle the promise using **.then()** method.
- The registered function receives the fulfillment value.
- **Eg to use a promise completely**:

```
newPromise.then(()=>{
      document.body.innerHTML = 'Promise consumed successfully'
})
```

Asynchronous task completed

Promise consumed succesfully

- Inside **resolve(),** we tend to pass a **promise value**, which can be a **single value**, an **object** or an **array**, etc.
- This **promise value** is **passed as a parameter to .then()**
- **Eg**

```
new Promise((resolve, reject)=>{
 setTimeout(() => {
  resolve({
            user_id: "angel_priya",
            password: "priya@146"
          })
    }, 3000);
  }).then((user)=>{
       console.log(user);
 })
```

Output at the console

{user_id: 'angel_priya', password: 'priya@146'}

- If you want to print a particular property of an object which is getting passed in resolve, then this can be achieved by chaining of .then()

```
new Promise((resolve, reject)=>{
  setTimeout(() => {
   resolve({
        user_id: "angel_priya",
        password: "priya@146"
   })
    }, 3000);
   }).then((user)=>{
        return user.user_id;
   }).then((userid)=>{
        console.log(userid);
   })
```

Output at the console will have value of user_id

angel_priya

Made with GAMMA

Asynchronous Promises

.catch()

- The **catch()** method registers a function that handles a rejected Promise
- **Eg**:

```
new Promise((resolve,reject)=>{
    setTimeout(() => {
      let error = false;
       if(!error){
         resolve({
           acc_no: 15324923430,
           ifsc: "BOBBKC"
         })
       }
      else{
          reject("ERROR, Can't fetch details")
           }
}, 4000);
   }).then((user)=>{
         console.log(user);
  }).catch((error)=>{
         console.log(error);
```

Output:

When Promise resolved

{acc_no: 15324923430, ifsc: 'BOBBKC'}

When Promise rejected

ERROR, Can't fetch details

.finally()

- The **finally**() method **registers a function that runs when a Promise settles**.
- It runs whether the Promise is fulfilled or rejected.
- **Eg**:

```
new Promise((resolve,reject)=>{
    setTimeout(() => {
      let error = false;
       if(!error){
         resolve({
           acc_no: 15324923430,
           ifsc: "BOBBKC"
         })
       }
  else{
      reject("ERROR, Can't fetch details")
      }
}, 4000);
   }).then((user)=>{
         console.log(user);
  }).catch((error)=>{
         console.log(error);
```

Output:

{acc_no: 15324923430, ifsc: 'BOBBKC'}

Promise succesfully settled

Made with GAMMA

Async and Await

- JavaScript also provides the **async** and **await** keywords, that **let's you write Promise-based code in a style that looks like synchronous JavaScript**.
- These keywords make Promise-based code easier to read.

Async Keyword

- The **async** keyword before a function **makes a function return a promise**.
- This is true even if you return a normal value.
- If the **function returns a value**, **JavaScript automatically wraps that value in a Promise.**
- **Eg :**

```
async function helloWorld(){
    return 'Hello world'
}
```

Above function **"helloWorld"** is a async function which is returning a promise.

Now, we have to handle the promise return by async function helloWorld.

```
helloWorld().then((value)=>{
    console.log(value);
})
```

After handling a promise by above given code, we would see **"hello world"** inside the console, **as the async function was returning "hello world" as a promise**.

Made with GAMMA

Async and Await

- As we saw in above example that any function when initated by an **async** keyword, **returns a promise**. That promise now is further handled along with the help of **await keyword**.

Await Keyword

- The **await** keyword **waits for an promise to be settled first**.
- It can **only be used inside an async function** or at the **top level of a JavaScript module**.
- While the **async function is waiting,** the rest of the program can continue running.
- The **await** keyword **pauses only the current async function**. **It does not pause the whole JavaScript**.
- Let's take an example of following promise and handle it using async and await.
- **Eg of a promise**

```
const pracPromise = new Promise((resolve, reject) => {
  setTimeout(() => {
    let error = false;

    if (!error) {
      resolve({
        prod_id: 111,
        prod_name: "Mobile cover",
      });
    } else {
      reject("ERROR, Product cannot be shown");
    }
  }, 2000);
})
```

Handling of promise using async and await.

```
// creating a async function
async function handlePracPromise(){
const pracPromiseResponse = await pracPromise
console.log(pracPromiseResponse);
}
// calling an async function
handlePracPromise();
```

Output at the console when no error is in the promise

{prod_id: 111, prod_name: 'Mobile cover'}

a. **prod_id**: 111
b. **prod_name**: "Mobile cover"
c. [[Prototype]]: Object

- In above case, our promise didn't had any error, **but in case of any error we have to handle using try and catch block inside a async function**.

Handling of promise having error using try and catch block in async and await

```
const pracPromise = new Promise((resolve, reject) => {
  setTimeout(() => {
    let error = true;

    if (!error) {
      resolve({
        prod_id: 111,
        prod_name: "Mobile cover",
      });
    } else {
      reject("ERROR, Product cannot be shown");
    }
  }, 2000);
})
```

```
async function handlePracPromise() {
 try {
const pracPromiseResponse = await pracPromise;
console.log(pracPromiseResponse.prod_name);
} catch (err) {
    console.log(err);
  }
}
```

Output from the console

ERROR, Product cannot be shown

Made with GAMMA